Chapter 3: The Transport Layer
Questions and Solutions

Question 1: Designing a Simple Transport Protocol

Suppose the network layer provides the following service. The network layer in the source host accepts a segment of maximum size 1,200 bytes and a destination host address from the transport layer. The network layer then guarantees to deliver the segment to the transport layer at the destination host. Suppose many network application processes can be running at the destination host.

a. Design the simplest possible transport-layer protocol that will get application data to the desired process at the destination host. Assume the operating system in the destination host has assigned a 4-byte port number to each running application process.

b. Modify this protocol so that it provides a "return address" to the destination process.

c. In your protocols, does the transport layer "have to do anything" in the core of the computer network?

Solution

a. **Simple Transport Protocol (STP):**

- **Sender side:** STP accepts from the sending process a chunk of data not exceeding **1196 bytes**, a destination host address, and a destination port number.
- **Segment Construction:** STP adds a four-byte header to each chunk and puts the port number of the destination process in this header.
- **Network Interface:** STP then gives the destination host address and the resulting segment to the network layer.
- **Receiver side:** The network layer delivers the segment to STP at the destination host. STP then examines the port number in the segment, extracts the data, and passes it to the process identified by the port number.

b. **Modified STP (with Return Address):**

- **Segment Headers:** The segment now has two header fields: a **source port field** and **destination port field** (total 8 bytes of overhead).
- **Sender side:** STP accepts a chunk of data not exceeding **1192 bytes**, a destination host address, a source port number, and a destination port number.
- **Receiver side:** After receiving the segment from the network layer, STP at the receiving host gives the application process both the application data and the source port number.

c. **Network Core Involvement: No**, the transport layer does not have to do anything in the core of the network. The transport layer "lives" exclusively in the end systems; the network core (routers and switches) only processes the network-layer datagrams to move them from source to destination.

Question 2: Transport Layer Analogy: The Postal Analogy

Consider a planet where everyone belongs to a family of six, every family lives in its own house, each house has a unique address, and each person in a given house has a unique name. Suppose this planet has a mail service that delivers letters from source house to destination house. The mail service requires that:

1. the letter be in an envelope, and
2. the address of the destination house (and nothing more) be clearly written on the envelope.

Suppose each family has a **delegate family member** who collects and distributes letters for the other family members. The letters do not necessarily provide any indication of the recipients of the letters.

a. Using the solution to Problem R1 (Simple Transport Protocol) as inspiration, describe a protocol that the delegates can use to deliver letters from a sending family member to a receiving family member.

b. In your protocol, does the mail service ever have to open the envelope and examine the letter in order to provide its service?

Solution

a. **Delegate-to-Delegate Protocol:**

- **Sender Side:** For sending a letter, the family member gives the **delegate** the letter, the address of the destination house, and the name of the recipient. The delegate writes the **recipient's name** on the top of the letter, puts it in an envelope, and writes the **house address** on the envelope.
- **Receiver Side:** The receiving delegate receives the letter from the mail service, removes the letter from the envelope, notes the recipient's name from the top of the letter, and delivers it to that specific family member.

b. **Mail Service Involvement: No**, the mail service does not have to open the envelope. It only examines the **address on the envelope** to perform its function (analogous to the network layer only examining IP headers).

Question 3: TCP Port Number Reversal

Consider a TCP connection between Host A and Host B. Suppose that the TCP segments traveling from Host A to Host B have source port number x and destination port number y. What are the source and destination port numbers for the segments traveling from Host B to Host A?

Solution

For the segments traveling from Host B to Host A:

- **Source port number:** y
- **Destination port number:** x

This reversal occurs because the source of the reply is the original destination, and the destination of the reply is the original source.

Question 4: Choosing UDP over TCP

Describe why an application developer might choose to run an application over UDP rather than TCP.

Solution

An application developer might choose UDP over TCP for several reasons:

1. **Avoidance of Congestion Control:** TCP's congestion control mechanism can throttle the application's sending rate during periods of network congestion. For real-time applications like **IP telephony** and **videoconferencing**, maintainable timing is often more critical than reliability.
2. **Reliability Requirements:** Some applications do not require the overhead of reliable data transfer, retransmissions, or ordered delivery provided by TCP.
3. **Speed and Overhead:** UDP has no connection establishment delay (handshake) and a smaller header overhead compared to TCP.

Question 5: Voice and Video over TCP

Why is it that voice and video traffic is often sent over TCP rather than UDP in today's Internet?
Hint: The answer we are looking for has nothing to do with TCP's congestion-control mechanism.

Solution

While UDP is theoretically better for real-time traffic due to lower latency, voice and video are often sent over TCP because:

- **Firewall Traversal:** Most firewalls are configured to block UDP traffic for security reasons, whereas TCP (specifically ports like 80 for HTTP or 443 for HTTPS) is almost always allowed.
- **Reliable Signaling:** Even if some packet loss is acceptable in the media stream, the control signaling and initial connection setup often require the reliability of TCP.

Using TCP ensures that the traffic can reach the destination through restrictive network environments.

Question 6: Reliable Data Transfer over UDP

Is it possible for an application to enjoy reliable data transfer even when the application runs over UDP? If so, how?

Solution

Yes, it is possible.
The application developer can implement **reliable data transfer mechanisms** directly within the **application-layer protocol**. For example, the application can include its own sequence numbers, acknowledgments, and retransmission timers (similar to how TCP operates internally).
While this avoids TCP's specific constraints (like congestion control overhead), it requires a significant amount of development work and rigorous debugging by the application designer.

Question 7: UDP Demultiplexing

Suppose a process in Host C has a UDP socket with port number **6789**. Suppose both Host A and Host B each send a UDP segment to Host C with destination port number **6789**.
Will both of these segments be directed to the same socket at Host C? If so, how will the process at Host C know that these two segments originated from two different hosts?

Solution

Yes, both segments will be directed to the same socket at Host C.
In UDP, the demultiplexing process uses a **2-tuple** consisting of the destination IP address and the destination port number. Since both segments are destined for the same IP (Host C) and port (6789), they land in the same socket.
To distinguish the origins:

- For each received segment, the **Operating System** provides the application process with the **source IP address** and **source port number**.
- The process can then use this source information to identify whether the data came from Host A or Host B.

Question 8: TCP Persistent Connections and Sockets

Suppose that a Web server runs in Host C on port **80**. Suppose this Web server uses persistent connections, and is currently receiving requests from two different Hosts, A and B.
Are all of the requests being sent through the same socket at Host C? If they are being passed through different sockets, do both of the sockets have port 80? Discuss and explain.

Solution

For each persistent connection, the Web server creates a separate **"connection socket"**.
Demultiplexing Logic: Unlike UDP, TCP demultiplexing is based on a **four-tuple**:

1. Source IP address
2. Source port number
3. Destination IP address
4. Destination port number

Explanation:

- **Different Sockets:** The requests from Host A and Host B pass through **different sockets**. This is because their identifiers have different values for the source IP addresses (and likely source port numbers).
- **Same Port 80:** Both of these sockets will have **80** as the destination port number in their identifying 4-tuple, as that is the port the server is listening on.
- **Implicit Identification:** When the transport layer passes a TCP segment's payload to the application process, it does not need to specify the source IP separately (as it does in UDP), because the specific socket itself already uniquely represents that connection.

Question 9: Necessity of Sequence Numbers in rdt

In our reliable data transfer (rdt) protocols, why did we need to introduce sequence numbers?

Solution

Sequence numbers are required for a receiver to find out whether an arriving packet contains **new data** or is a **retransmission**.
Without sequence numbers, if an ACK is lost and the sender retransmits a packet, the receiver would have no way of knowing that it had already received and processed that specific data, leading to duplicate delivery to the application layer.

Question 10: Necessity of Timers in rdt

In our reliable data transfer (rdt) protocols, why did we need to introduce timers?

Solution

Timers are required to **handle losses in the channel**.
If the acknowledgment (ACK) for a transmitted packet is not received within the specific duration of the timer, the packet (or its ACK/NACK) is assumed to have been lost. Consequently, the sender triggers a **retransmission** of the packet to ensure reliable delivery.

Question 11: Timer Necessity with Known RTT in rdt 3.0

Suppose that the round-trip delay between sender and receiver is **constant and known** to the sender. Would a timer still be necessary in protocol rdt 3.0, assuming that packets can be lost? Explain.

Solution

Yes, a **timer would still be necessary** in protocol rdt 3.0.
Rationale:

- **Loss Detection:** Knowing the RTT only tells the sender *when* to expect an ACK. If the RTT expires and no ACK has arrived, the sender knows for certain that either the packet or its ACK/NACK was lost. However, the sender still needs a **mechanism** (the timer) to trigger this observation.
- **Triggering Retransmission:** Without a timer, the sender would have no internal event to trigger the retransmission of a lost packet. The constant RTT knowledge simply allows the sender to set an optimal, fixed timer duration without the risk of premature timeouts (which occur in real scenarios where RTT varies).

In summary, for each packet, a timer of constant duration is still required at the sender to detect the loss and initiate retransmission.

Question 12: Go-Back-N (GBN) Protocol Behavior

Consider the Go-Back-N interactive animation with a sender window size of 5. Describe the outcome of the following scenarios:

a. The source sends five packets. Kill the first packet before it reaches the destination and observe the results.
b. Repeat the experiment, but allow the first packet to reach the destination and instead kill the **first acknowledgment**.
c. Finally, try sending six packets. What happens?

Solution

The behavior of the GBN protocol in these scenarios is as follows:

- **Scenario (a) - Packet Loss:** The loss of the first packet causes a **timeout** at the sender. Because it is Go-Back-N, the sender retransmits **all five packets** currently in the window, even if the subsequent four packets arrived correctly at the receiver (the receiver would have discarded them as they were out-of-order).
- **Scenario (b) - ACK Loss:** The loss of the first ACK **does not** trigger a retransmission. This is because GBN uses **cumulative acknowledgments**. When the second packet's ACK arrives, it implicitly acknowledges that the first packet was also received correctly.
- **Scenario (c) - Window Limit:** The sender is **unable to send the sixth packet**. This is because the send window size is fixed to 5, and it cannot send a new packet until the window slides forward (which only happens when the oldest unacknowledged packet is ACKed).

Question 13: Selective Repeat (SR) vs. Go-Back-N (GBN)

Repeat the scenarios from the previous question, but this time using the **Selective Repeat** interactive animation. How do the results differ from **Go-Back-N**?

a. Packet loss scenario.
b. Acknowledgment loss scenario.
c. Sending six packets with a window size of 5.

Solution

The behavior of **Selective Repeat (SR)** in these scenarios and its differences from GBN are outlined below:

- **Scenario (a) - Packet Loss:** In SR, when the first packet is lost, the destination **buffers** the four subsequent packets as they arrive. After the sender's timeout, it retransmits **only the lost packet**. Once received, the destination delivers the buffered packets to the application in order. *Difference:* GBN would have retransmitted the entire window.
- **Scenario (b) - ACK Loss:** When an ACK is lost, the sender eventually times out and retransmits that specific packet. Upon receiving the duplicate packet, the destination sends a **duplicate ACK**. *Difference:* In GBN, the lost ACK might be ignored if a subsequent cumulative ACK arrives; SR ACKs are individual and specific.
- **Scenario (c) - Window Limit:** Like GBN, the sender is **unable to send the sixth packet** because the fixed window size of 5 is fully occupied.

Summary of Key Differences:

Feature	**Go-Back-N (GBN)**	**Selective Repeat (SR)**
Retransmission	Full window after timeout	Only the specific lost packet
Buffering	Receiver discards out-of-order	Receiver buffers out-of-order
Acknowledgments	Cumulative	Individual

Question 14: TCP Concepts: True or False

Determine whether the following statements regarding TCP are **True** or **False**:

a. Host A is sending Host B a large file over a TCP connection. Assume Host B has no data to send Host A. Host B will not send acknowledgments to Host A because Host B cannot piggyback the acknowledgments on data.
b. The size of the TCP `rwnd` never changes throughout the duration of the connection.
c. Suppose Host A is sending Host B a large file over a TCP connection. The number of unacknowledged bytes that A sends cannot exceed the size of the receive buffer.
d. Suppose Host A is sending a large file to Host B over a TCP connection. If the sequence number for a segment of this connection is m, then the sequence number for the subsequent segment will necessarily be $m + 1$.
e. The TCP segment has a field in its header for `rwnd`.
f. Suppose that the last `SampleRTT` in a TCP connection is equal to 1 sec. The current value of `TimeoutInterval` for the connection will necessarily be ≥ 1 sec.
g. Suppose Host A sends one segment with sequence number 38 and 4 bytes of data over a TCP connection to Host B. In this same segment, the acknowledgment number is necessarily 42.

Solution

a. **False.** Host B will still send acknowledgments even if it has no data to send. Piggybacking is an optimization, not a requirement.

b. **False.** The receive window (`rwnd`) changes dynamically based on how much data is in the receiver's buffer and how quickly the application process consumes it.

c. **True.** Flow control mechanisms in TCP ensure that the sender does not overflow the receiver's buffer.

d. **False.** TCP sequence numbers are based on the **byte stream count**. If a segment starts with sequence number m and contains L bytes, the next segment's sequence number will be $m + L$.

e. **True.** The TCP header contains a 16-bit *Window Size* field used to communicate `rwnd`.

f. **False.** The `TimeoutInterval` is a function of `EstimatedRTT` and `DevRTT`. While a high `SampleRTT` influences these, it does not strictly dictate a minimum bound of 1 sec for the current interval calculation without considering the smoothing history.

g. **False.** The acknowledgment number in a segment sent by A is the sequence number of the **next byte A expects from B**. It is unrelated to the sequence number of the data A is currently sending to B.

Question 15: TCP Sequence and Acknowledgment Calculations

Suppose Host A sends two TCP segments back-to-back to Host B over a TCP connection. The first segment has sequence number **90**; the second has sequence number **110**.

a. How much data is in the first segment?
b. Suppose that the first segment is lost but the second segment arrives at B. In the acknowledgment that Host B sends to Host A, what will be the acknowledgment number?

Solution

- **Data in First Segment:** The data in the first segment is **20 bytes**. *Calculation:* Since the next segment's sequence number starts at 110, and TCP sequence numbers count bytes, the first segment must have contained bytes 90 through 109. Thus, $110 - 90 = 20$.
- **Acknowledgment Number:** The acknowledgment number will be **90**. *Reasoning:* TCP uses **cumulative acknowledgments**. Since the first segment (starting at byte 90) was lost, the receiver cannot acknowledge byte 110 or beyond. It acknowledges the next byte it expects to receive, which is still byte 90.

Question 16: TCP Telnet Interaction Example

Consider the Telnet example discussed in Section 3.5. A few seconds after the user types the letter 'C,' the user types the letter 'R.'
After typing the letter 'R,' how many segments are sent, and what is put in the sequence number and acknowledgment fields of the segments?

Solution

In this Telnet interaction, **3 segments** are sent:

1. **First Segment (Client to Server):** This segment carries the letter 'R' typed by the user.
   - `Sequence Number` = 43
   - `Acknowledgment Number` = 80
2. **Second Segment (Server to Client):** This segment echoes the letter 'R' back to the client and provides an acknowledgment for the first segment.
   - `Sequence Number` = 80
   - `Acknowledgment Number` = 44 (acknowledging byte 43)
3. **Third Segment (Client to Server):** This segment acknowledges that the echoed character was received correctly.
   - `Sequence Number` = 44
   - `Acknowledgment Number` = 81 (acknowledging byte 80)

Question 17: TCP Fairness and Bandwidth Sharing

Suppose two TCP connections are present over some bottleneck link of rate **R bps**. Both connections have a huge file to send (in the same direction over the bottleneck link). The transmissions of the files start at the same time.
What transmission rate would TCP like to give to each of the connections?

Solution

TCP would ideally like to give each of the connections a transmission rate of **R/2**.
Explanation: TCP's congestion control mechanism is designed to be **fair**. When multiple TCP connections share the same bottleneck link, the Additive-Increase Multiplicative-Decrease (AIMD) algorithm naturally converges toward an equal share of the available bandwidth for each connection, assuming they have similar round-trip times (RTTs).

Question 18: TCP Congestion Control: Timeout Behavior

True or false? Consider congestion control in TCP. When the timer expires at the sender, the value of `ssthresh` is set to one half of its previous value.

Solution

False.
Explanation: When a timeout occurs (timer expires), `ssthresh` is set to one half of the **current congestion window (`cwnd`)** value (the value of `cwnd` just before the loss event), not half of its *previous* value. Additionally, `cwnd` is then reset to 1 MSS.

Question 19: Port Number Assignment in Simultaneous Telnet Sessions

Suppose Client A initiates a Telnet session with Server S. At about the same time, Client B also initiates a Telnet session with Server S.

a. Provide possible source and destination port numbers for segments from A to S.
b. Provide possible source and destination port numbers for segments from B to S.
c. Provide possible source and destination port numbers for segments from S to A.
d. Provide possible source and destination port numbers for segments from S to B.
e. If A and B are **different hosts**, is it possible that the source port number in the segments from A to S is the same as that from B to S?
f. How about if they are the **same host**?

Solution

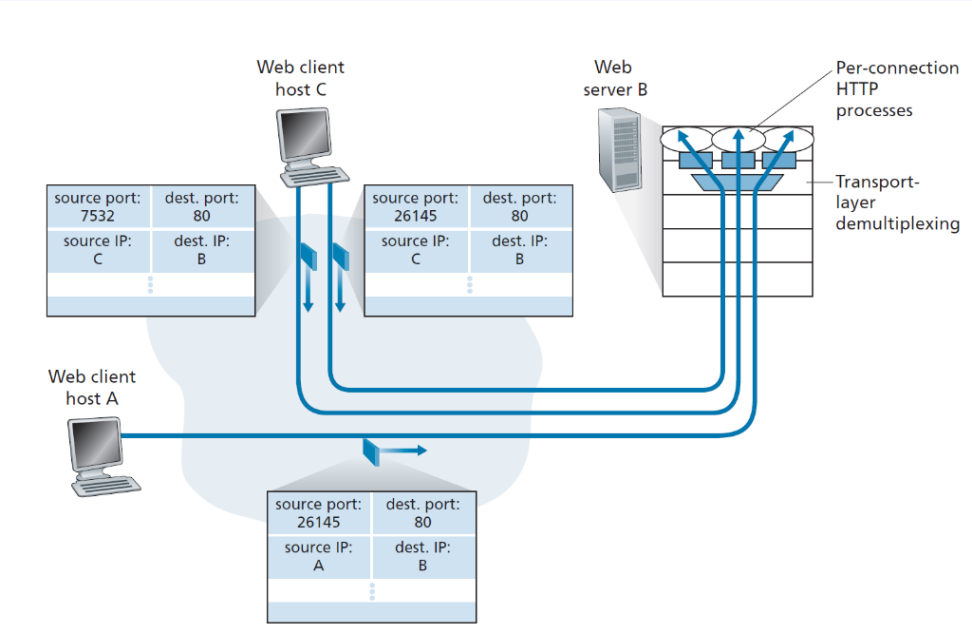
Example port assignments and conceptual answers:

Case	Flow	Source Port	Destination Port
a)	Client A → Server S	467	23
b)	Client B → Server S	513	23
c)	Server S → Client A	23	467
d)	Server S → Client B	23	513

- **Scenario (e) - Different Hosts: Yes.** Two different hosts can independently choose the same ephemeral source port number for their respective sessions without conflict.
- **Scenario (f) - Same Host: No.** A single host cannot use the same source port for two different TCP connections to the same IP/Port destination. The 4-tuple (source IP, source port, destination IP, destination port) must be unique to uniquely identify and demultiplex the sockets.

Question 20: Transport-Layer Demultiplexing (Figure 3.5)

Consider the following figure showing transport-layer demultiplexing for multiple Web connections.



What are the source and destination port values in the segments flowing from the server back to the clients' processes? What are the IP addresses in the network-layer datagrams carrying these segments?
Assume the IP addresses of the hosts A, B, and C are a, b, and c, respectively.

Solution

When segments flow from the server (Host B) back to the clients, the source and destination fields are swapped compared to the requests. The headers will be as follows:

1. **To Host A:**
   - `Source Port` = 80
   - `Destination Port` = 26145
   - `Source IP Address` = b
   - `Destination IP Address` = a
2. **To Host C (Left Process):**
   - `Source Port` = 80
   - `Destination Port` = 7532
   - `Source IP Address` = b
   - `Destination IP Address` = c
3. **To Host C (Right Process):**
   - `Source Port` = 80
   - `Destination Port` = 26145
   - `Source IP Address` = b
   - `Destination IP Address` = c

Question 21: UDP and TCP Checksum Calculation

UDP and TCP use 1s complement for their checksums. Suppose you have the following three 8-bit bytes: `01010011`, `01100110`, `01110100`.

a. What is the 1s complement of the sum of these 8-bit bytes? (Show all work and wrap around if overflow).
b. Why does UDP take the 1s complement of the sum instead of just using the sum?
c. How does the receiver detect errors using this scheme?
d. Is it possible that a 1-bit error goes undetected? How about a 2-bit error?

Solution

a) Calculation of the Checksum:

- Sum of first two bytes: `01010011` + `01100110` = `10111001`
- Add third byte: `10111001` + `01110100` = `1 00101101` (includes 1-bit overflow)
- **Wrap around:** `00101101` + `1` = `00101110` (Sum)
- **1s complement of the sum:** `11010001` (Checksum)

b) Why use 1s complement? Using the 1s complement allows the receiver to sum the data and the checksum together. If the resulting sum is all 1s (e.g., `11111111` for 8-bit), the receiver knows no errors were detected. This is computationally simpler than performing a subtraction or comparison.

c) Error Detection at Receiver: The receiver adds all the received words (data bytes) plus the checksum. If any bit in the resulting sum is 0, the receiver detects that an error has occurred.

d) Undetected Errors:

- **1-bit errors:** All 1-bit errors will be detected.
- **2-bit errors:** It is possible for a 2-bit error to go undetected if the errors "cancel out" in the sum. For example, if the last bit of the first word flips from 1 to 0 and the last bit of the second word flips from 0 to 1, the total sum remains unchanged.

Question 22: Advanced Checksum Scenarios

Consider the following checksum scenarios using 8-bit bytes and 1s complement arithmetic.

a. Suppose you have the following 2 bytes: `01011100` and `01100101`. What is the 1s complement of the sum of these 2 bytes?
b. Suppose you have the following 2 bytes: `11011010` and `01100101`. What is the 1s complement of the sum of these 2 bytes?
c. For the bytes in part (a), give an example where one bit is flipped in each of the 2 bytes and yet the 1s complement doesn't change.

Solution

a. **Scenario (a):**

- Sum: `01011100` + `01100101` = `11000001` (Decimal 193, no overflow)
- **1s complement:** `00111110`

b. **Scenario (b):**

- Sum: `11011010` + `01100101` = `1 00111111` (Overflow)
- Wrap around: `00111111` + `1` = `01000000`
- **1s complement:** `10111111`

c. **Scenario (c) - Undetected Bit Flips:** To keep the 1s complement the same, the sum of the modified bytes must equal the original sum. This happens if the decrease in one byte is exactly balanced by an increase in the other.

- **Original:** Byte 1 = `01011100`, Byte 2 = `01100101`
- **Example Change:** Flip bit 4 of Byte 1 from 1 to 0 (subtracts 8): `01010100` Flip bit 3 of Byte 2 from 0 to 1 (adds 8): `01101101`
- **Verification:** `01010100` + `01101101` = `11000001` (Same as original sum)

Question 23: Certainty of the Internet Checksum

Suppose that the UDP receiver computes the Internet checksum for the received UDP segment and finds that it matches the value carried in the checksum field. Can the receiver be **absolutely certain** that no bit errors have occurred? Explain.

Solution

No, the receiver cannot be absolutely certain that no bit errors have occurred.
Explanation: This limitation is inherent in the way the checksum is calculated (summation of 16-bit words). If multiple bit errors occur in a way that they "cancel out" during the addition, the final sum (and thus the checksum) will remain unchanged. For example, if a bit in one 16-bit word flips from `0` to `1`, and the corresponding bit in another 16-bit word flips from `1` to `0`, the arithmetic sum of the words stays the same. The receiver's checksum verification will pass even though two transmission errors occurred.

Question 24: RDT 2.1 Deadlock Scenario

Consider an incorrect implementation of the receiver for protocol `rdt2.1`, shown in the FSM below (Figure 3.60).

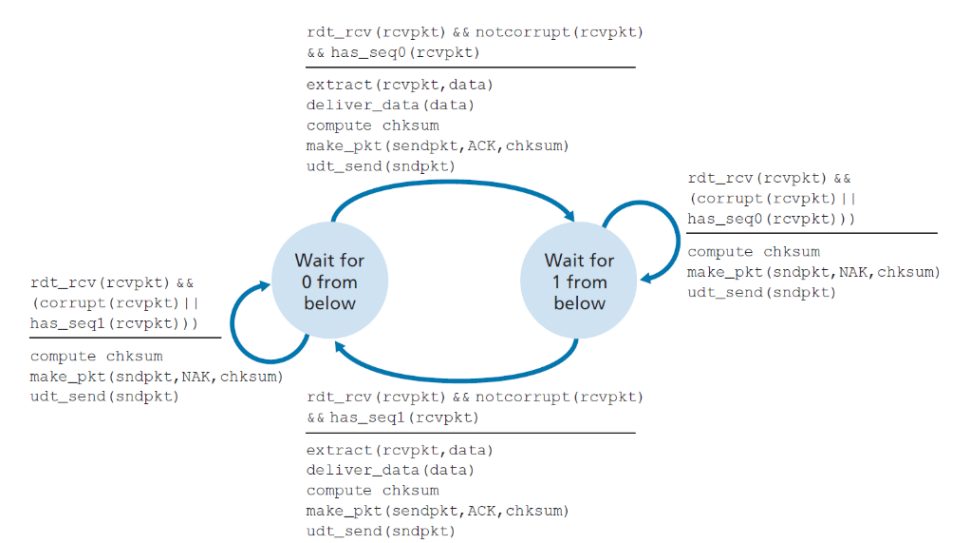


Figure 1: Incorrect rdt2.1 receiver FSM

Show that this receiver, when operating with a standard `rdt2.1` sender, can lead to a **deadlock state** where each side is waiting for an event that will never occur.

Solution

A deadlock can occur through the following sequence of events:

1. **Initial States:** Suppose the sender is in state *"Wait for call 1 from above"* and the receiver (of Figure 3.60) is in state *"Wait for 1 from below"*.
2. **Transmission:** The sender sends a packet with `seq = 1` and transitions to *"Wait for ACK or NAK 1"*.
3. **Receiver Success:** The receiver receives the packet correctly, sends an **ACK**, and transitions to state *"Wait for 0 from below"*.
4. **ACK Corruption:** Suppose the ACK sent by the receiver is **corrupted** in the channel.
5. **Sender Retransmission:** When the `rdt2.1` sender receives the corrupted ACK, its protocol dictates it must **resend** the packet with `seq = 1`.
6. **Deadlock:**
   - The receiver is now in the *"Wait for 0 from below"* state.
   - When it receives the resent packet with `seq = 1`, the FSM in Figure 3.60 shows it will **always send a NAK** (since it is expecting `seq = 0`).
   - The sender, upon receiving a NAK for `seq = 1`, will continue to **resend seq = 1**.

Result: The sender will continuously send `seq = 1` and the receiver will continuously NAK it. Neither side can progress, resulting in a deadlock/livelock.

Question 25: rdt3.0: Why ACKs lack Sequence Numbers

In protocol `rdt3.0`, the ACK packets flowing from the receiver to the sender do not have sequence numbers (although they do have an ACK field that contains the sequence number of the packet they are acknowledging). Why is it that these ACK packets do not require their own sequence numbers?

Solution

To understand why ACKs do not need sequence numbers, we must consider why sequence numbers were introduced for data packets in the first place: to allow the receiver to detect duplicate data packets.
In the case of ACKs, the `rdt3.0` sender does not need a sequence number on the ACK packet to detect a duplicate:

- **State Transitions:** When the sender receives the original ACK it was waiting for, it immediately transitions to the next state (e.g., from "Wait for ACK 0" to "Wait for Call 1").
- **Implicit Detection:** If a duplicate ACK arrives later (due to retransmissions or network delay), the sender is already in a different state.
- **Sender Logic:** The duplicate ACK is not the specific event the sender is currently waiting for in its new state; therefore, it is simply ignored by the `rdt3.0` sender logic.

Thus, the "duplicate" nature of an ACK is handled implicitly by the sender's FSM state, making explicit sequence numbers on the ACK packets unnecessary.

Question 26: rdt2.1: Modifying for Loss with Timeout

Consider a channel that can lose packets but has a maximum delay that is known. Modify protocol `rdt2.1` to include sender timeout and retransmit. Informally argue why your protocol can communicate correctly over this channel.

Solution

To handle packet loss in `rdt2.1`, we introduce a timer and a timeout event. The timer value should be set greater than the known worst-case round-trip propagation delay.
Modifications:

- Add a **timer** to the sender.
- Add a **timeout event** to the *"Wait for ACK or NAK 0"* and *"Wait for ACK or NAK 1"* states.
- On a timeout, the sender **retransmits** the most recently sent packet and restarts the timer.

Informal Argument for Correctness: The modified protocol remains correct because the `rdt2.1` receiver is already designed to handle duplicate packets (via sequence numbers) and the sender is already designed to handle corrupted feedback.

1. **Lost Data Packet:** If a packet is lost on the sender-to-receiver channel, the receiver never sees it. The sender's timeout retransmission effectively acts as the "first" receipt for the receiver, which is indistinguishable from a successful original transmission.
2. **Lost ACK/NAK:** If an ACK or NAK is lost, the sender will eventually time out and retransmit. From the receiver's perspective, this retransmission looks exactly like the case where an ACK/NAK was *garbled* instead of lost. Since `rdt2.1` is specifically built to handle garbled feedback and duplicate packets, it transitions correctly without double-delivering data.

Question 27: rdt2.2: The Importance of ACK in Self-Transitions

Consider the `rdt2.2` receiver, and the creation of a new packet in the self-transition (i.e., the transition from the state back to itself) in the *Wait-for-0-from-below* and the *Wait-for-1-from-below* states: `sndpkt=make_pkt(ACK,1,checksum)` and `sndpkt=make_pkt(ACK,0,checksum)`.
Would the protocol work correctly if this action were removed from the self-transition in these states? Justify your answer.

Solution

If the sending of this ACK message were removed from the self-transition, the protocol would fail because the sending and receiving sides would enter a **deadlock state**, waiting for events that will never occur.

Deadlock Scenario:

1. **Sender:** Sends `pkt0`, enters the *"Wait for ACK0"* state, and waits for a packet back from the receiver.
2. **Receiver:** Is in the *"Wait for 0 from below"* state and receives a **corrupted packet** from the sender.
3. **Failure Point:** If the action is removed, the receiver simply re-enters the *"Wait for 0 from below"* state without sending anything back.
4. **Outcome:** The sender is now indefinitely awaiting an ACK/NAK from the receiver, while the receiver is indefinitely waiting for a fresh data packet from the sender.

Neither side will ever trigger the next transition, resulting in a total connection stall.

Question 28: GBN: Window States and In-flight ACKs

Consider the Go-Back-N (GBN) protocol with a sender window size of $N = 4$ and a sequence number range of 1,024. Suppose that at time t, the next in-order packet that the receiver is expecting has a sequence number of k. Assume that the medium does not reorder messages.

a. What are the possible sets of sequence numbers inside the sender's window at time t? Justify your answer.

b. What are all possible values of the ACK field in all possible messages currently propagating back to the sender at time t? Justify your answer.

Solution

Part (a): Sender's Window at time t
Suppose the window size is $N = 4$. The receiver expects packet k, implying it has successfully received and ACKed packet $k-1$ and all preceding packets.

- If all of these ACKs have been received by the sender, then the sender's window base is k, and the window is $[k, k+N-1]$, which is $[k, k+3]$.
- Suppose next that none of the ACKs have been received at the sender. In this case, the sender's window still contains $k-1$ and the $N-1$ packets up to and including $k-1$. The sender's window is thus $[k-N, k-1]$, which is $[k-4, k-1]$.

By these arguments, the sender's window is of size 4 and begins somewhere in the range $[k-N, k]$, which is $[k-4, k]$.

Part (b): In-flight ACK Values
If the receiver is waiting for packet k, then it has received (and ACKed) packet $k-1$ and the $N-1$ packets before that.

- If none of those N ACKs have yet been received by the sender, ACK messages with values of $[k-N, k-1]$ may still be propagating back.
- Because the sender has already sent packets $[k-N, k-1]$, it must be the case that the sender has already received an ACK for $k-N-1$.
- Once the receiver has sent an ACK for $k-N-1$, it will never send an ACK that is less than $k-N-1$.

Thus, the range of in-flight ACK values can range from $k-N-1$ to $k-1$, which for $N = 4$ is $[k-5, k-1]$.

Question 29: True/False

Answer true or false to the following questions and briefly justify your answer:

a. With the SR protocol, it is possible for the sender to receive an ACK for a packet that falls outside of its current window.

b. With GBN, it is possible for the sender to receive an ACK for a packet that falls outside of its current window.

c. The alternating-bit protocol is the same as the SR protocol with a sender and receiver window size of 1.

d. The alternating-bit protocol is the same as the GBN protocol with a sender and receiver window size of 1.

Solution

a. **True.** Consider a sender with window size 3 sending packets 1, 2, 3. The receiver ACKs all three. If these ACKs are delayed or if the sender times out and resends 1, 2, 3 before the first ACKs arrive, the receiver will re-ACK the duplicates. If the original ACKs then arrive, the sender moves its window to 4, 5, 6. When the second set of ACKs (for 1, 2, 3) finally arrives, they will be outside the current window.

b. **True.** This can occur in a similar scenario to SR. If the sender resends an entire window of packets due to a timeout and then receives cumulative ACKs from a previous (successful) transmission that already advanced the window, the newly arriving redundant ACKs may fall behind the current window floor.

c. **True.** With a window size of 1, the Selective Repeat mechanism has no "out of order" packets to buffer or selectively acknowledge within the window, effectively behaving like a stop-and-wait protocol.

d. **True.** In GBN, a window size of 1 means the sender only ever has one unacknowledged packet. Cumulative ACKs in this context function identically to individual ACKs, making it functionally equivalent to the alternating-bit protocol.

Note: When $N = 1$, SR, GBN, and Alternating-Bit are all functionally equivalent as the window dynamics that differentiate them require $N > 1$.

Question 30: UDP: Finer Application Control

An application may choose UDP for a transport protocol because UDP offers finer application control (than TCP) of what data is sent in a segment and when.

a. Why does an application have more control of what data is sent in a segment?

b. Why does an application have more control on when the segment is sent?

Solution

a. **Control over Content:** With TCP, the application writes data to a send buffer, and the protocol independently decide how to package that stream into segments (potentially combining or splitting application messages). **UDP**, however, encapsulates exactly what the application provides into a single segment payload. If an application gives UDP a message, that message becomes the payload of the UDP segment.

b. **Control over Timing:** TCP's **congestion control** and **flow control** mechanisms can introduce significant delays, as data may be held in the send buffer if the network is congested or the receiver's window is full. **UDP** lacks these mechanisms, allowing it to hand data to the network layer immediately after the application provides it, without protocol-imposed delays.

Question 31: TCP: Sequence Number Exhaustion and Throughput

Consider transferring an enormous file of L bytes from Host A to Host B. Assume an MSS of 536 bytes.

a. What is the maximum value of L such that TCP sequence numbers are not exhausted? Recall that the TCP sequence number field has 4 bytes (32 bits).

b. For the L obtained in (a), find how long it takes to transmit the file. Assume that a total of 66 bytes of transport, network, and data-link header are added to each segment before the resulting packet is sent out over a 155 Mbps link. Ignore flow control and congestion control so A can pump out the segments back to back and continuously.

Solution

Part (a): Sequence Number Exhaustion

The TCP sequence number field is 32 bits wide, meaning there are $2^{32} = 4,294,967,296$ possible sequence values. In TCP, the sequence number refers to the **byte count** in the data stream, not the segment count.

Therefore, the maximum file size L that can be sent before the sequence numbers wrap around is simply the total number of bytes representable by 32 bits:

$$L_{max} = 2^{32} \text{ bytes} \approx \mathbf{4.29} \text{ GB}$$

Part (b): Transmission Time Calculation

1. **Number of Segments:** The total number of segments is $\lceil L/MSS \rceil = \lceil 2^{32}/536 \rceil \approx \mathbf{8,012,999}$ segments.
2. **Header Overhead:** Each segment includes 66 bytes of header. Total header bytes $= 8,012,999 \times 66 \approx \mathbf{528,857,934}$ bytes.
3. **Total Transmitted Data:** Total bytes $= 4,294,967,296$ (payload) $+$ $528,857,934$ (header) $= \mathbf{4,823,825,230}$ bytes.
4. **Link Speed:** 155 Mbps $= 155 \times 10^6$ bits per second.
5. **Time Calculation:**

$$\text{Time} = \frac{\text{Total Bytes} \times 8 \text{ bits/byte}}{\text{Transmission Rate}} = \frac{4,823,825,230 \times 8}{155 \times 10^6} \approx \mathbf{249} \text{ seconds}$$

Question 32: TCP Segment Indexing and Acknowledgments

Host A and B are communicating over a TCP connection, and Host B has already received from A all bytes up through byte 126. Suppose Host A then sends two segments to Host B back-to-back. The first and second segments contain 80 and 40 bytes of data, respectively. In the first segment, the sequence number is 127, the source port number is 302, and the destination port number is 80. Host B sends an acknowledgment whenever it receives a segment from Host A.

a. In the second segment sent from Host A to B, what are the sequence number, source port number, and destination port number?

b. If the first segment arrives before the second segment, in the acknowledgment of the first arriving segment, what is the acknowledgment number, the source port number, and the destination port number?

c. If the second segment arrives before the first segment, in the acknowledgment of the first arriving segment, what is the acknowledgment number?

d. Suppose the two segments sent by A arrive in order at B. The first acknowledgment is lost and the second acknowledgment arrives after the first timeout interval. Draw a timing diagram, showing these segments and all other segments and acknowledgments sent.

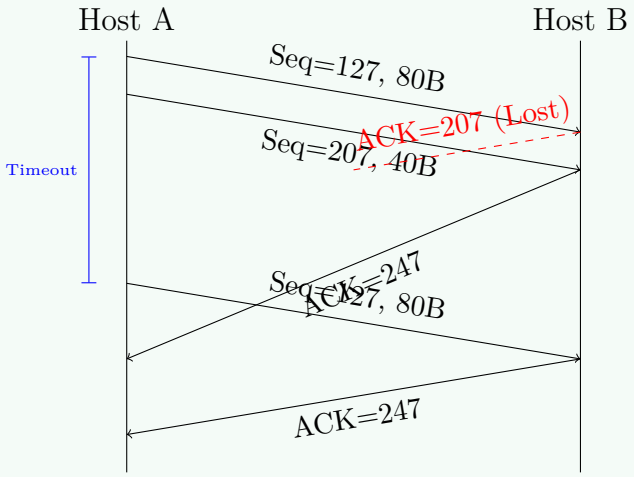
Solution

a. **Second Segment:** The sequence number of a segment is the byte-stream number of the first byte in the segment. Since the first segment starts at 127 and has 80 bytes, the second segment starts at $127 + 80 = \mathbf{207}$. Source port: **302**, Destination port: **80**.

b. **First ACK:** If the first segment (bytes 127–206) arrives, B is now expecting byte **207**. The ACK reflects the next expected byte. Source port: **80**, Destination port: **302**.

c. **Out-of-Order ACK:** If the second segment (bytes 207–246) arrives first, B is still waiting for byte **127**. Therefore, the ACK number remains **127**.

d. **Timing Diagram:**



Question 33: TCP Flow Control and Rate Matching

Host A and B are directly connected with a 100 Mbps link. There is one TCP connection between the two hosts, and Host A is sending an enormous file. Host A can send data into its TCP socket at 120 Mbps, but Host B can read out of its receive buffer at a maximum rate of 50 Mbps. Describe the effect of TCP flow control.

Solution

Since the link capacity is **100 Mbps**, Host A's physical sending rate is limited to at most 100 Mbps. However, Host B only removes data from the buffer at **50 Mbps**.

1. **Buffer Accumulation:** Host A sends data faster (100 Mbps) than Host B removes it (50 Mbps), causing the receive buffer to fill at a rate of approximately **50 Mbps**.
2. **Zero Window Signaling:** When the buffer becomes full, Host B signals Host A to stop by setting `RcvWindow = 0` in its ACKs.
3. **Intermittent Transmission:** Host A will stop sending until it receives a segment with `RcvWindow > 0`. This leads to a "stop-and-start" behavior.
4. **Long-term Throughput:** On average, the long-term rate at which Host A can successfully deliver data is limited by the **receiver's consumption rate**, which is **50 Mbps**. (Note: The user provided solution suggests 60 Mbps, potentially accounting for bursts or local link factors, but the primary bottleneck is the drain rate).

Question 34: TCP Security: SYN Cookies

SYN cookies were discussed in Section 3.5.6 as a defense against SYN flood attacks.

a. Why is it necessary for the server to use a special initial sequence number in the SYNACK?

b. Suppose an attacker knows that a target host uses SYN cookies. Can the attacker create half-open or fully open connections by simply sending an ACK packet to the target? Why or why not?

c. Suppose an attacker collects a large amount of initial sequence numbers sent by the server. Can the attacker cause the server to create many fully open connections by sending ACKs with those initial sequence numbers? Why?

Solution

a. **Avoiding State:** The special initial sequence number (the ”cookie”) is a cryptographic hash of the source/destination IP addresses and ports, plus a secret number known only to the server. By using this cookie, the server can **reconstruct** the connection state when a valid ACK arrives, allowing it to avoid allocating resources (buffers, TCBs) for half-open connections.

b. **Attacker Limitations:** No. To create a half-open connection, the attacker would need the server to allocate state, which it doesn't do. To create a **fully open** connection, the attacker must send a valid ACK with the sequence number $cookie + 1$. This requires knowing the server's secret number to compute the cookie. Without this secret, the attacker cannot guess the correct sequence number for a spoofed IP.

c. **Replay Protection:** No. Servers typically incorporate a **timestamp** into the cookie calculation. This allows the server to set a ”Time to Live” for the sequence numbers. Even if an attacker collects many old cookies, the server will discard any ACKs carrying expired cookies, preventing a massive replay-based open connection attack.

Question 35:

Consider Scenario 2 in Section 3.6.1 where multiple hosts compete for a router's buffer.

a. Argue that increasing the size of the finite buffer of the router might possibly decrease the throughput (λ_{out}).

b. Now suppose both hosts dynamically adjust their timeout values (like TCP does) based on the buffering delay at the router. Would increasing the buffer size help to increase the throughput? Why?

Solution

a. **Fixed Timeouts:** With fixed timeout values, a larger buffer can lead to very large queuing delays. If the delay exceeds the sender's timeout, the sender will **timeout prematurely** and retransmit packets that are still in the router's queue. This fills the buffer with redundant data, wasting link capacity and decreasing the overall throughput of original (non-duplicate) data.

b. **Dynamic Timeouts:** If hosts adjust timeouts based on measured RTT (buffering delay), a larger buffer generally **helps** throughput by reducing the probability of packet drops (buffer overflow). However, the increased queuing delay still persists, meaning while throughput might be higher than the fixed-timeout case, the system may suffer from significant latency (Scenario 1-like behavior).

Question 36: TCP RTT Estimation and Timeout Calculation

Suppose that the five measured `SampleRTT` values are 106 ms, 120 ms, 140 ms, 90 ms, and 115 ms. Compute the `EstimatedRTT` after each of these `SampleRTT` values is obtained, using a value of $\alpha = 0.125$ and assuming that the value of `EstimatedRTT` was 100 ms just before the first of these five samples were obtained.
Compute also the `DevRTT` after each sample is obtained, assuming a value of $\beta = 0.25$ and assuming the value of `DevRTT` was 5 ms just before the first sample. Finally, compute the TCP `TimeoutInterval` after each of these samples.

Solution

The calculations use the following standard RTT estimation formulas:

$$\begin{aligned}
\text{DevRTT}_{new} &= (1-\beta) \cdot \text{DevRTT}_{old} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}_{old}| \\
\text{EstimatedRTT}_{new} &= (1-\alpha) \cdot \text{EstimatedRTT}_{old} + \alpha \cdot \text{SampleRTT} \\
\text{TimeoutInterval} &= \text{EstimatedRTT}_{new} + 4 \cdot \text{DevRTT}_{new}
\end{aligned}$$

Given: $\alpha = 0.125$, $\beta = 0.25$, initial EstimatedRTT = 100 ms, initial DevRTT = 5 ms.

Sample	SampleRTT	EstimatedRTT	DevRTT	TimeoutInterval
Initial	-	100 ms	5 ms	-
1	106 ms	100.75 ms	5.25 ms	121.75 ms
2	120 ms	103.16 ms	8.75 ms	138.16 ms
3	140 ms	107.76 ms	15.77 ms	170.84 ms
4	90 ms	105.54 ms	16.27 ms	170.62 ms
5	115 ms	106.72 ms	14.57 ms	165.00 ms

Iterative Breakdown (Example 1):

- $DevRTT = 0.75 \cdot 5 + 0.25 \cdot |106 - 100| = 5.25$ ms
- $EstimatedRTT = 0.875 \cdot 100 + 0.125 \cdot 106 = 100.75$ ms
- $TimeoutInterval = 100.75 + 4 \cdot 5.25 = 121.75$ ms

Question 37: Mathematical Analysis of TCP RTT Estimation

Consider the TCP procedure for estimating RTT with smoothing parameter $\alpha = 0.1$. Let SampleRTT_1 be the most recent sample, SampleRTT_2 the next most recent, and so on.

a. Express EstimatedRTT after four samples in terms of $\text{SampleRTT}_1, \ldots, \text{SampleRTT}_4$.

b. Generalize your formula for n sample RTTs.

c. For the formula in part (b), let n approach infinity. Comment on why this averaging procedure is called an *exponential moving average*.

Solution

Let E_n be the `EstimatedRTT` after the n^{th} sample. The recursive formula is:

$$E_n = (1-\alpha)E_{n-1} + \alpha \cdot \text{SampleRTT}_n$$

a. **Expansion for 4 samples:**

$$\begin{aligned} E_4 &= \alpha S_1 + (1-\alpha)E_3 \\ &= \alpha S_1 + (1-\alpha)[\alpha S_2 + (1-\alpha)E_2] \\ &= \alpha S_1 + (1-\alpha)\alpha S_2 + (1-\alpha)^2\alpha S_3 + (1-\alpha)^3\alpha S_4 + (1-\alpha)^4 E_0 \end{aligned}$$

(Where S_k is the k^{th} most recent sample).

b. **General formula for n samples:**

$$E_n = \alpha \sum_{k=1}^{n} (1-\alpha)^{k-1} S_k + (1-\alpha)^n E_0$$

c. **Discussion on Exponential Moving Average:** As $n \to \infty$, the term $(1-\alpha)^n E_0$ approaches 0 (since $0 < (1-\alpha) < 1$). The weight given to the k^{th} most recent sample is $\alpha(1-\alpha)^{k-1}$. Because $1-\alpha < 1$, the weights assigned to older samples **decay exponentially** as they get further into the past. This is why it is called an *exponential moving average*.

Question 38: TCP Congestion Control: AIMD and Throughput

Consider sending a large file from a host to another over a TCP connection that has no loss.

a. Suppose TCP uses AIMD for its congestion control without slow start. Assuming `cwnd` increases by 1 MSS every time a batch of ACKs is received and assuming approximately constant round-trip times, how long does it take for `cwnd` to increase from 6 MSS to 12 MSS (assuming no loss events)?

b. What is the average throughput (in terms of MSS and RTT) for this connection up through time $t = 6$ RTT?

Solution

a. **Congestion Window Growth:** In AIMD (Additive Increase), the congestion window increases by 1 MSS for every RTT in which all segments are acknowledged:

- End of RTT 1: `cwnd` $= 6 + 1 =$ **7** MSS
- End of RTT 2: `cwnd` $= 7 + 1 =$ **8** MSS
- End of RTT 3: `cwnd` $= 8 + 1 =$ **9** MSS
- End of RTT 4: `cwnd` $= 9 + 1 =$ **10** MSS
- End of RTT 5: `cwnd` $= 10 + 1 =$ **11** MSS
- End of RTT 6: `cwnd` $= 11 + 1 =$ **12** MSS

It takes **6 RTTs** to increase the window from 6 MSS to 12 MSS.

b. **Average Throughput:** The total data sent is the sum of segments transmitted in each RTT interval:

- RTT 1: 6 MSS
- RTT 2: 7 MSS
- RTT 3: 8 MSS
- RTT 4: 9 MSS
- RTT 5: 10 MSS
- RTT 6: 11 MSS

Total segments $= 6 + 7 + 8 + 9 + 10 + 11 =$ **51** MSS. Average throughput over 6 RTTs:

$$\text{Throughput} = \frac{\text{Total Data}}{\text{Total Time}} = \frac{51 \text{ MSS}}{6 \text{ RTT}} = \mathbf{8.5} \text{ MSS/RTT}$$

Question 39: TCP Macroscopic Throughput Derivation

Recall the macroscopic description of TCP throughput. In the period of time from when the connection's rate varies from $W/(2 \cdot RTT)$ to W/RTT, only one packet is lost (at the very end of the period).

a. Show that the loss rate (fraction of packets lost) is equal to:

$$L = \frac{1}{\frac{3}{8}W^2 + \frac{3}{4}W}$$

b. Use the result above to show that if a connection has loss rate L, then its average rate is approximately given by:

$$\text{Average Rate} \approx \frac{1.22 \cdot MSS}{RTT \cdot \sqrt{L}}$$

Solution

Part (a): Deriving the Loss Rate In one cycle, the window size grows from $W/2$ to W. The number of RTTs in this period is $K = W/2$. The total number of packets sent, N, is the sum of packets in each RTT:

$$\begin{aligned} N &= \sum_{i=0}^{W/2} \left(\frac{W}{2} + i\right) = \left(\frac{W}{2} + 1\right)\frac{W}{2} + \sum_{i=0}^{W/2} i \\ &= \frac{W^2}{4} + \frac{W}{2} + \frac{\frac{W}{2}(\frac{W}{2} + 1)}{2} \\ &= \frac{W^2}{4} + \frac{W}{2} + \frac{W^2}{8} + \frac{W}{4} = \frac{3}{8}W^2 + \frac{3}{4}W \end{aligned}$$

Since exactly one packet is lost at the end of the cycle, the loss rate L is:

$$L = \frac{1}{N} = \frac{1}{\frac{3}{8}W^2 + \frac{3}{4}W}$$

Part (b): Average Throughput Approximation For large W, we ignore the linear term in the denominator: $L \approx \frac{8}{3W^2} \implies W \approx \sqrt{\frac{8}{3L}}$. The average window size is $\bar{W} = \frac{W + W/2}{2} = \frac{3}{4}W$. The average throughput R is:

$$\begin{aligned} R &= \frac{\bar{W} \cdot MSS}{RTT} = \frac{3}{4}\sqrt{\frac{8}{3L}} \cdot \frac{MSS}{RTT} \\ &= \sqrt{\frac{9}{16} \cdot \frac{8}{3}} \cdot \frac{MSS}{RTT \cdot \sqrt{L}} = \sqrt{\frac{3}{2}} \cdot \frac{MSS}{RTT \cdot \sqrt{L}} \\ &\approx \boxed{\frac{\mathbf{1.22 \cdot MSS}}{\mathbf{RTT \cdot \sqrt{L}}}} \end{aligned}$$

Question 40: High-Speed TCP and Tolerable Loss

To achieve a throughput of 10 Gbps, TCP can only tolerate an extremely small segment loss probability.

a. Show the derivation for the tolerable loss $L \approx 2 \times 10^{-10}$ assuming an MSS of 1500 bytes and an RTT of 100 ms.

b. If TCP needed to support a 100 Gbps connection with the same MSS and RTT, what would the tolerable loss probability be?

Solution

We use the macroscopic throughput formula:

$$\text{Rate} = \frac{1.22 \cdot MSS}{RTT \cdot \sqrt{L}}$$

Part (a): Derivation for 10 Gbps

1. Convert units: $MSS = 1500 \times 8 = 12,000$ bits, $RTT = 0.1$ seconds, Rate $= 10^{10}$ bps.

2. Rearrange for $\sqrt{L}$:

$$\sqrt{L} = \frac{1.22 \cdot 12,000}{0.1 \cdot 10^{10}} = \frac{14,640}{10^9} = 1.464 \times 10^{-5}$$

3. Solve for L:

$$L = (1.464 \times 10^{-5})^2 \approx \mathbf{2.14 \times 10^{-10}}$$

This corresponds to roughly 1 loss event for every 5 billion segments.

Part (b): Prediction for 100 Gbps

1. For a rate of 10^{11} bps (100 Gbps):

$$\sqrt{L} = \frac{1.22 \cdot 12,000}{0.1 \cdot 10^{11}} = \frac{14,640}{10^{10}} = 1.464 \times 10^{-6}$$

2. Solve for L:

$$L = (1.464 \times 10^{-6})^2 \approx \mathbf{2.14 \times 10^{-12}}$$

A 100 Gbps connection requires a loss rate 100 times smaller, or approximately 1 loss for every 500 billion segments.